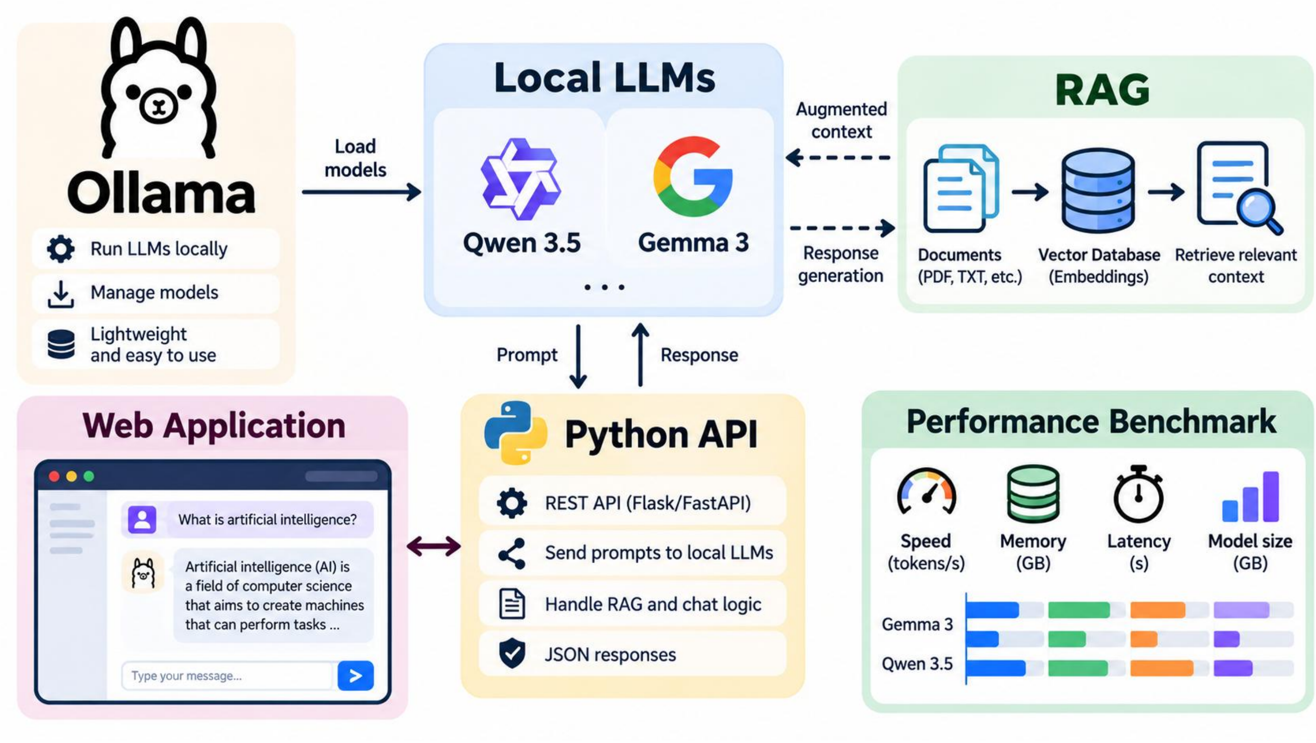
كلية العلوم و التقنيات بطنجة
ⵜⴰⵖⵉⵡⴰⵏⵜ ⵏ ⵜⵎⴰⵙⵙⴰⵏⵉⵏ ⴷ ⵜⵙⵏⵉⵅⵉⵜⵉⵏ ⵏ ⵟⴰⵏⵊⴰ
Faculté des Sciences et Techniques de Tanger

جامعة عبد المالك السعدي
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵏ ⵄⴰⴱⴷⵍⵎⴰⵍⴽ ⵙⵙⴰⵄⴷⵉ
Université Abdelmalek Essaâdi

Local AI with Ollama

Lab Report — Local LLMs, RAG, Python API, Web Application and Performance Benchmar


Ollama
Run LLMs locally
Manage models
Lightweight and easy to use
Load models
Local LLMs
Qwen 3.5
Gemma 3
...
Augmented context
Response generation
RAG
Documents (PDF, TXT, etc.)
Vector Database (Embeddings)
Retrieve relevant context
Prompt
Response
Web Application
What is artificial intelligence?
Artificial intelligence (AI) is a field of computer science that aims to create machines that can perform tasks ...
Type your message...
Python API
REST API (Flask/FastAPI)
Send prompts to local LLMs
Handle RAG and chat logic
JSON responses
Performance Benchmark
Speed (tokens/s)
Memory (GB)
Latency (s)
Model size (GB)
Gemma 3
Qwen 3.5

Presented by: Fatima STITOU

Supervised by: Pr. Ikram BENABDELOUAHAB
and Pr. Lotfi EL AACHAK

1. Introduction

This lab explores how to run large language models (LLMs) locally using Ollama, connect them to Python, compare their performance, build a simple chatbot and web application, and use local documents as a knowledge source through Retrieval-Augmented Generation (RAG). The experiments were performed on a Windows computer using the locally installed models Qwen 3.5 and Gemma 3.

2. Questions for the Report

Q1. What is Local AI?

Local AI means running an AI model directly on a personal computer or local server instead of sending every request to a remote cloud service. The model and inference process run on the user's own hardware.

Q2. Difference between a Cloud LLM and a Local LLM?

A cloud LLM runs on a provider's remote servers and is accessed over the internet. A local LLM runs on the user's machine. Cloud models may offer more computing power and easier scaling, while local models offer more control over data and can work without an internet connection after installation.

Q3. What role does Ollama play?

Ollama manages local models: downloading them, running inference, loading and unloading them, and exposing an API that applications can call. In this project, Python sends prompts to Ollama and receives the generated responses.

Q4. What is model quantization?

Quantization stores model weights using lower-precision numerical representations, such as 4-bit values instead of higher-precision values. It usually reduces model size and memory use and can improve local inference speed, although it may slightly reduce response quality.

Q5. Why does model size affect memory consumption?

The model's parameters must be loaded into memory during inference. A model with more parameters or higher-precision weights generally requires more RAM or GPU memory. Context length and the prompt also require additional memory for the model's working state and cache.

Q6. Why can a smaller model sometimes be preferable?

A smaller model can load faster, use less memory, and generate answers more quickly on limited hardware. If its answers are sufficiently accurate for the task, it may be more practical than a larger model that overloads the computer.

Q7. Advantages of local inference?

Local inference can improve data control and privacy, reduce dependence on internet connectivity, avoid per-request cloud API charges, and allow experimentation with models and prompts on the user's own machine.

Q8. Disadvantages of local inference?

Local inference depends on the computer's available RAM, CPU, and GPU. Models may be slower or less capable than larger cloud models, setup and maintenance are the user's responsibility, and exposing a local server carelessly can create security risks.

Q9. Explain the architecture of your Python application.

The application has a browser-based interface served by Flask. The Flask/Python backend receives the user's question, optionally retrieves relevant passages from local course documents using a RAG component, builds the prompt or message history, and sends a request to Ollama. Ollama runs the selected local model and returns a response, which the backend sends back to the browser. The lab also includes a console chatbot and benchmark code.

Q10. Explain the role of the REST API.

The REST API provides HTTP endpoints through which Python communicates with Ollama. For example, the application sends a POST request to /api/generate or /api/chat with the model name and prompt/messages, and Ollama returns a JSON response. This separates the application interface from the model runtime.

Q11. What is RAG?

Retrieval-Augmented Generation (RAG) retrieves relevant information from a document collection and supplies it to the LLM as context before generating an answer. In this lab, local course documents are searched so the assistant can answer from the supplied material and report when the information is unavailable.

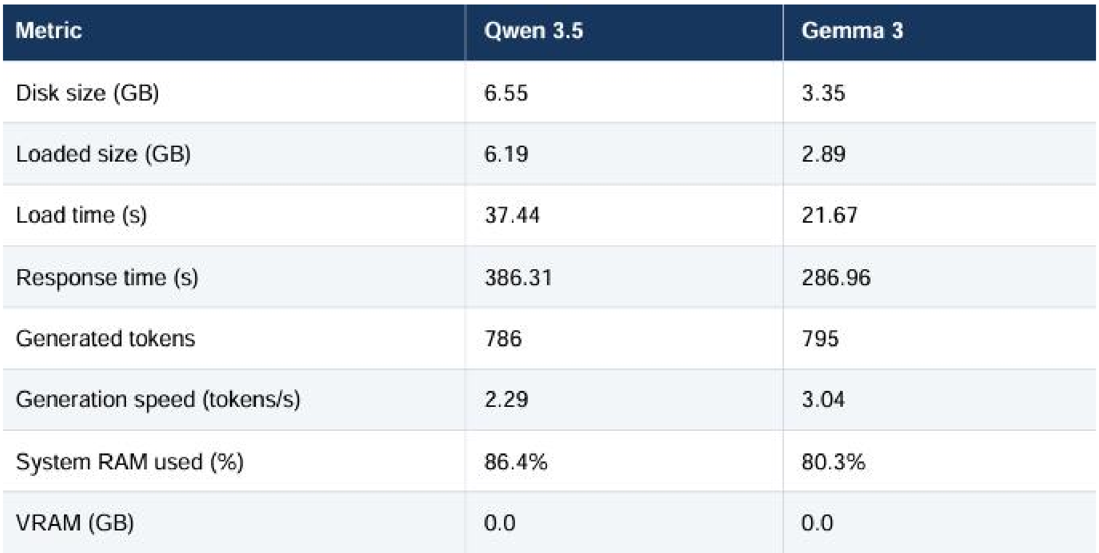
Q12. Difference between an LLM and a multimodal LLM?

A standard LLM primarily processes text. A multimodal LLM can work with more than one type of input or output, such as text and images. In the vision exercise, a vision-capable model such as Gemma 3 is given an image encoded as Base64 together with a prompt asking it to describe the image.

3. Experimental Comparison

Q13. Compare two local models experimentally.

Both models were tested locally through Ollama. The figures below use the measured values shown in the benchmark results in the notebook screenshot. They describe this computer and this test prompt; they are not universal rankings.



Metric	Qwen 3.5	Gemma 3
Disk size (GB)	6.55	3.35
Loaded size (GB)	6.19	2.89
Load time (s)	37.44	21.67
Response time (s)	386.31	286.96
Generated tokens	786	795
Generation speed (tokens/s)	2.29	3.04
System RAM used (%)	86.4%	80.3%
VRAM (GB)	0.0	0.0

Interpretation: Gemma 3 was smaller on disk and in loaded memory, loaded faster, had a shorter total response time, generated more tokens per second, and showed lower system RAM usage in this run. The RAM values can vary between runs; they should be interpreted alongside the other metrics. The separate programming exercise scored Qwen 3.5 at 4/5 for response quality and 5/5 for code correctness, and Gemma 3 at 3/5 and 4/5 respectively, with response times of 433.4 s and 396.5 s. These scores apply to that particular coding prompt, not to all tasks.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

I would choose Gemma 3 for this computer because the measured loaded size was lower (2.89 GB compared with 6.19 GB for Qwen 3.5), and it used less system RAM in the benchmark screenshot (80.3% compared with 86.4%). It also generated tokens faster in that run. However, the computer's RAM usage was still high, so I would close unnecessary applications or use a smaller/ more heavily quantized model if memory became a problem. I would also verify answer quality on the intended tasks before deployment.

4. 4. Security Considerations

Q15. What security risks exist when exposing a local LLM server to a network?

Ollama does not provide authentication by default. If its API is exposed to a network, an unauthorized person may send requests, consume CPU/RAM, or access model capabilities. Prompts and documents may contain private information, and malicious instructions hidden in retrieved documents can cause prompt-injection risks. The server should normally remain bound to localhost unless network access is necessary. If remote access is required, restrict it with a firewall and trusted network rules, and place it behind a reverse proxy that enforces authentication and HTTPS/TLS. Do not expose the API directly to the public internet.

5. Conclusion

This lab demonstrated a local AI workflow using Ollama, Python, REST requests, a chatbot, a Flask web interface, document retrieval, and a multimodal model. The measured benchmark favored Gemma 3 for model size, load time, response latency, and generation speed on this machine. The programming exercise showed that Qwen 3.5 scored higher for the specific code-generation task. Therefore, model selection should depend on the task, available hardware, measured performance, and response quality rather than model size alone.